

How Remi's terms relate mathematically

A walkthrough with a three-axis toy example · S1 ONTOLOGY · 2026-09-16

0. The one idea everything rests on

Every sensory word is a coordinate axis. Remi has 216 lexicon words, so a product's sensory state is a point in a 216-dimensional space. To keep the example readable we use **three axes**:

```
L-space (toy)   axis 1 = L.ta.sweet      axis 2 = L.tx.thickness    axis 3 = L.ta.sour

a score vector  s = ( s_sweet , s_thick , s_sour )      each entry is a ±3 panel score
the origin      o = ( 0 , 0 , 0 )                      = the benchmark: "same as reference"
```

That is why the field is called `axis`: an `L` id names one coordinate.

1. Node → vector: what one ingredient "is" to the model

An ingredient node has an id and nothing else. What the model uses is its **vector**: one number per axis, "how far a typical dose moves the score". In YAML we store only the non-zero entries, each as a **component** record:

```
ING.xanthan_gum
effects:
- {axis: L.tx.thickness, direction: increase,
  magnitude: strong}      # +1.0
- {axis: L.ta.sweet,      direction: decrease,
  magnitude: weak}        # -0.3 (gum masks sweetness)
```

the same thing as a vector

```
v(xanthan) = ( -0.3 , +1.0 , 0 )
              sweet  thick  sour
```

words → numbers (loader):
weak 0.3 · medium 0.6 · strong 1.0
decrease → minus sign

`component` one entry, e.g. "+1.0 on thickness" — the record carries its axis, sign, size, scope, confidence. `vector` the whole list = the ingredient's row of numbers. A zero is simply an axis that is not listed.

2. Roles: where most of an ingredient's vector comes from

Ingredients rarely list all their effects themselves. They inherit the vector of each **role** (FT) they belong to, scaled by the role weight, and then override what is special to them:

$$v(ING) = \sum \text{over roles } w_{\text{role}} \cdot v(FT_{\text{role}}) + \text{overrides}$$

```
sugar : roles = [ FT.sweetener (strong) ]
v(FT.sweetener) = ( +1.0 , +0.3 , 0 )      sweeteners add sweetness and a little body
v(sugar)        = 1.0 · ( +1.0 , +0.3 , 0 ) = ( +1.0 , +0.3 , 0 )
```

```
xanthan : roles = [ FT.thickener (strong) ]
v(FT.thickener) = ( 0 , +1.0 , 0 )
override on sweet: -0.3
v(xanthan)      = ( -0.3 , +1.0 , 0 )
```

This is the INGREDIENT layer's 2-tier design: shared numbers live on the role, exceptions on the ingredient. Role weights stay words (weak/medium/strong); learned numbers land on the ingredient's own components (rule A10).

3. Vectors stacked = the matrix Γ

Put one ingredient per row and one axis per column. That table is Γ — the thing the model actually multiplies.

Γ	sweet	thick	sour
sugar	+1.0	+0.3	0
xanthan	-0.3	+1.0	0

matrix is never written in YAML. It is what you get by stacking every ingredient's vector; the loader builds it. Real size: ~118 ingredients $\times$ 216 axes.

4. The three multiplications

4a. Predict — stage D1: "if I change the recipe, how do the scores move?"

The recipe change is a vector $\mathbf{x}$ with one entry per ingredient, in units of "typical doses added". Add one dose of sugar and half a dose of xanthan:

```
x = ( +1.0 , +0.5 )           (sugar, xanthan)

Δs = xT Γ                     row-vector × matrix

Δs_sweet = 1.0·(+1.0) + 0.5·(-0.3) = +0.85
Δs_thick = 1.0·(+0.3) + 0.5·(+1.0) = +0.80
Δs_sour  = 1.0·( 0 ) + 0.5·( 0 ) = 0

Δs = ( +0.85 , +0.80 , 0 )     "a bit sweeter, noticeably thicker, sourness unchanged"
```

Each score is a sum down one column: every ingredient's contribution to that axis, weighted by how much of it you added. This is the whole of "build a theoretical recipe" — the optimiser just searches for the $\mathbf{x}$ whose Δs is closest to the target.

4b. Interact — R-3: "perceptions also shift each other"

Some axes change other axes on their own — sweetness suppresses sourness. Those are the **IN** scalars, entries of an axis-by-axis matrix M . Our toy has one:

M	→ sweet	→ thick	→ sour
sweet →	0	0	-0.6
thick →	0	0	0
sour →	0	0	0

```
s' = s + M s

sour' = 0 + (-0.6)·(+0.85) = -0.51

s' = ( +0.85 , +0.80 , -0.51 )
"...and it will taste a little less sour"
```

scalar one **IN** record = one cell of M . The matrix is almost empty on purpose (rule C4: only well-established causal interactions).

4c. Measure — stage E1: "what would the lab see, and does it agree?"

A parameter is a lab reading that stands in for some axes. Its `axes` list is a vector over L ; stacked, they form Π . Viscosity stands in for thickness:

Π	sweet	thick	sour
P.apparent_viscosity	0	+1.0	0
P.pH	0	0	-1.0

$$m = \Pi s$$

`m_viscosity` = +0.80 (expected reading, scaled)
`m_pH` = +0.51 (less sour → higher pH)

if the lab reads viscosity $\approx$ +0.8:
confidence in `s_thick` goes up (rule A4)
if it reads something else:
the `xanthan→thick` component is corrected (E1)

The model runs on Δs alone; Π is only used when a reading exists. That is what "P is auxiliary" means in equations.

5. The tuple switches components on and off

The same ingredient does not behave the same in every product. Each component carries a **scope**; the profile tuple decides which components are active, so Γ is really $\Gamma(t)$:

```
xanthan → thickness : {scope: SC.emulsion.ow|APP.sauce,    magnitude: strong} → +1.0  
                      {scope: SC.emulsion.ow|APP.beverage,  magnitude: medium} → +0.6
```

```
t = [SC.emulsion.ow, APP.sauce,    ST.ambient] →  $\Gamma_t$ [xanthan, thick] = +1.0  
t = [SC.emulsion.ow, APP.beverage, ST.ambient] →  $\Gamma_t$ [xanthan, thick] = +0.6
```

matching rule (B5): a scope is "on" when every part it writes equals the profile's part;
a blank part matches anything. If two scopes match, the one with more parts written wins (B6).

tuple the profile `[SC, APP, ST]` is not a number — it is a selector. It never enters the multiplication; it chooses which numbers do.

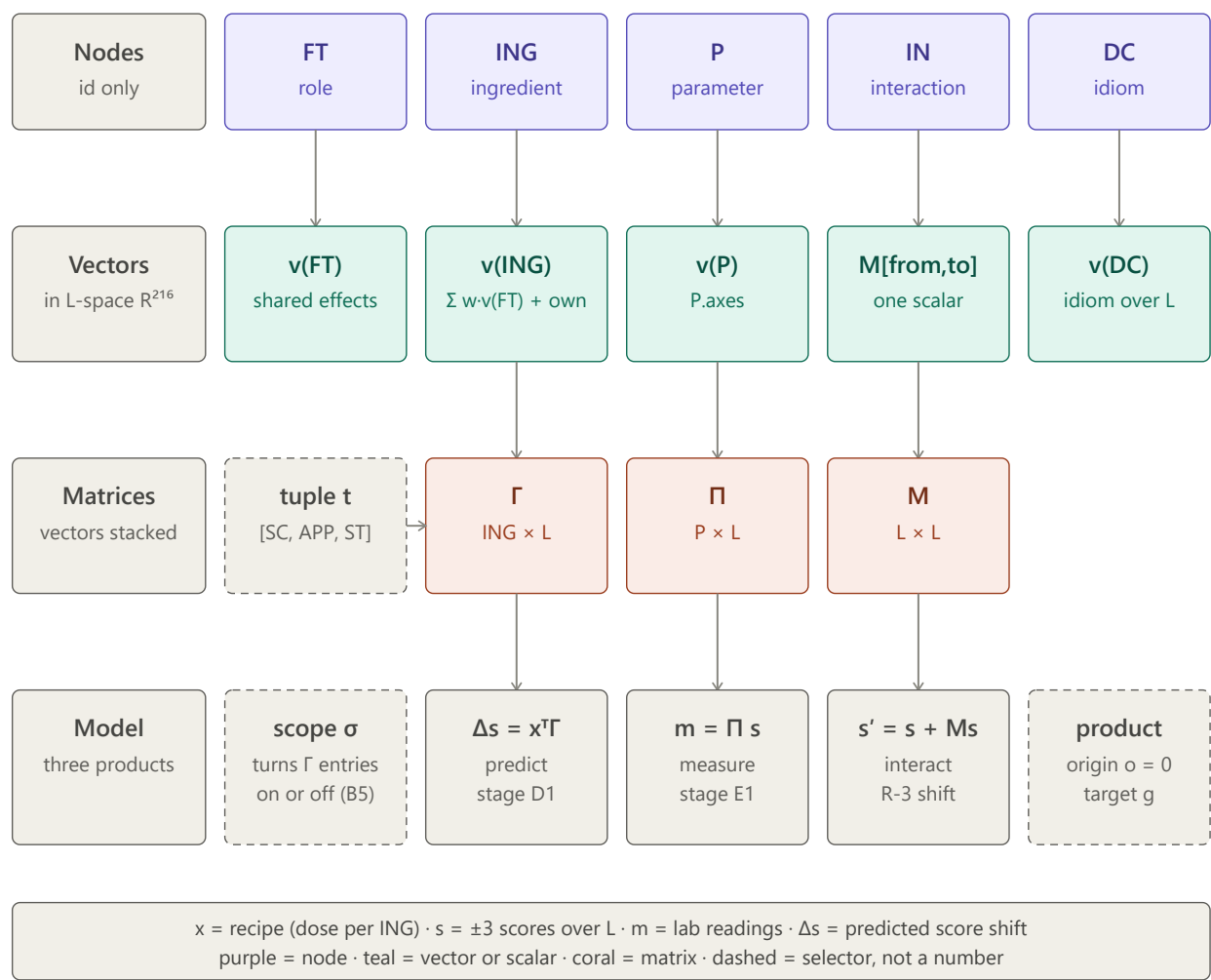
6. Where the product sits

```
product = ( t , key ingredients , identity axes  $I \subset L$  , origin o )
```

`o` = the reference sample; all scores are 0 there
`g` = the goal vector the user asked for, e.g. `g = ( -1 , +1 , 0 )` "less sweet, thicker"
D1 searches for `x` such that $\Delta s(x) \approx g$ subject to ranges and the palette

Two products with the same tuple but different key ingredients have different Γ rows in play and different origins — that is the "same ruler" rule (B9): scores are only comparable when `t`, key ingredients and `o` all match.

7. The map



Read one column at a time: **ING** owns $v(ING)$; those vectors stacked are Γ ; Γ is what the model multiplies by the recipe. **FT** and **DC** stop at the vector row — a role's vector is only inherited, an idiom's vector is a target direction. Dashed boxes are selectors, not numbers.

8. Glossary, tied to the example

Word	In the example	In the files
node	"sugar", "thickness", "viscosity" — names	an id line: <code>ING.sucrose</code> , <code>L.tx.thickness</code> , <code>P.apparent_viscosity</code>
component	+1.0 on thickness	one <code>{axis, direction, magnitude, ...}</code> record
vector	$v(\text{xanthan}) = (-0.3, +1.0, 0)$	the <code>effects</code> list of one ingredient (or <code>axes</code> of one P, one DC)
scalar	$M[\text{sweet}, \text{sour}] = -0.6$	one IN record
matrix	Γ , M , Π tables above	not stored; built by the loader from all vectors
tuple	[SC.emulsion.ow, APP.sauce, ST.ambient]	a profile entry; a scope string on a component
sentence	"gum masks sweetness — measured 2026-08"	a LESSON (observation) or RP (prescription)